

# Docker: Containerizing a Spring Boot Application & Small Base Images

## 1. The Practical Problem

Consider a simple Spring Boot application containing a single REST API.

HelloController.java

```
@RestController
public class HelloController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello from Docker!";
    }
}
```

Our goal is to package this application into a Docker image and run it inside a container.

### Step 1: Package the application

Run the following command from the project directory:

```
mvn clean package -DskipTests
```

Maven generates an executable JAR inside the `target` directory.

Start the application locally:

```
java -jar target/docker-demo-0.0.1-SNAPSHOT.jar
```

Access the API:

```
http://localhost:8080/hello
```

Expected response:

```
Hello from Docker!
```

The application is working locally. We can now containerize it.

## 2. Creating Our First Dockerfile

Create a file named `Dockerfile` in the root directory of the Spring Boot project.

```
FROM eclipse-temurin:21-jdk

WORKDIR /app

COPY target/docker-demo-0.0.1-SNAPSHOT.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

### Understanding the Dockerfile

| Instruction             | Purpose                                                                   |
|-------------------------|---------------------------------------------------------------------------|
| <code>FROM</code>       | Uses an existing image containing Java 21 JDK as the base.                |
| <code>WORKDIR</code>    | Sets <code>/app</code> as the working directory inside the image.         |
| <code>COPY</code>       | Copies our locally generated JAR into the image as <code>app.jar</code> . |
| <code>EXPOSE</code>     | Documents that the application listens on port 8080.                      |
| <code>ENTRYPOINT</code> | Defines the command executed when the container starts.                   |

Important: `EXPOSE` does not automatically publish a container's port to the host machine. We must explicitly publish the port using `-p` when starting the container.

### Step 2: Build the Docker image

```
docker build -t docker-demo:v1 .
```

This creates an image named `docker-demo` with the tag `v1`.

### Step 3: Run the container

```
docker run -d \
  --name demo-v1 \
  -p 8080:8080 \
  docker-demo:v1
```

Here:

- `-d` runs the container in detached mode.
- `--name` assigns a name to the container.
- `-p 8080:8080` maps host port 8080 to container port 8080.
- `docker-demo:v1` specifies the image to run.

Verify the application:

```
curl http://localhost:8080/hello
```

Expected response:

```
Hello from Docker!
```

Our Spring Boot application is now running inside a Docker container.

## 3. Understanding Small Base Images

Check the size of the image we just created:

```
docker images
```

Although the application contains only one REST API, its Docker image may be significantly larger than the actual application JAR.

To understand why, we need to examine what goes inside a Docker image.

## What is inside a base image?

A containerized Java application requires three essential components:

```
Linux User-Space Environment
+
  Java Runtime
+
  Application JAR
=
Runnable Docker Image
```

When we specify:

```
FROM eclipse-temurin:21-jdk
```

Docker uses an image containing both a Linux user-space environment and the Java Development Kit.

Our image has the following conceptual structure:

```
docker-demo:v1
├── Linux User-Space Environment
│   ├── Libraries
│   ├── Utilities
│   └── System Files
├── Java Development Kit (JDK)
│   ├── JVM
│   ├── Java Compiler
│   └── Development Tools
└── Spring Boot Application
    └── app.jar
```

Note: A Docker image does not contain a complete operating system with its own kernel. Containers share the host's kernel.

## Do we really need the JDK?

Consider the command used to start our application:

```
ENTRYPOINT ["java","-jar","app.jar"]
```

We are executing an already compiled JAR.

The Java compiler and other development tools are not required for the normal execution of this application.

A JRE (Java Runtime Environment) provides the runtime components necessary to execute Java applications without including the complete set of JDK development tools.

Using a JRE-based image instead of a full JDK image can reduce the size of our final Docker image.

## Why do smaller images matter?

Smaller Docker images offer several practical advantages:

- Reduced image download and transfer times.
- Lower storage requirements in image registries and deployment environments.
- Potentially faster image distribution during deployments.
- A smaller software footprint, which may reduce unnecessary components and their associated vulnerabilities.

However, image size alone does not guarantee security. The underlying packages must still be maintained and scanned for vulnerabilities.

## 4. Practical: Replacing JDK with JRE

We can optimize our existing image by changing just one instruction.

### Dockerfile v2

```
FROM eclipse-temurin:21-jre

WORKDIR /app

COPY target/docker-demo-0.0.1-SNAPSHOT.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

The only modification is:

```
- FROM eclipse-temurin:21-jdk
+ FROM eclipse-temurin:21-jre
```

Everything else remains unchanged.

## Step 1: Build the optimized image

```
docker build -t docker-demo:v2 .
```

## Step 2: Compare image sizes

```
docker images
```

Compare the sizes of both versions:

| Image          | Base Image  | Expected Result |
|----------------|-------------|-----------------|
| docker-demo:v1 | Java 21 JDK | Larger image    |
| docker-demo:v2 | Java 21 JRE | Smaller image   |

The actual size difference depends on the base-image version and platform.

## Step 3: Run the optimized container

```
docker run -d --name demo-v2 -p 8081:8080 \
  docker-demo:v2
```

Verify:

```
curl http://localhost:8081/hello
```

Expected response:

```
Hello from Docker!
```

The optimized container produces exactly the same output as the previous version.

## Understanding the port mapping

We used:

```
-p8081:8080
```

The format is:

```
HOST_PORT : CONTAINER_PORT
```

In our example:

| Host Machine   |        | Docker Container                     |
|----------------|--------|--------------------------------------|
| localhost:8081 | -----> | Port 8080<br>Spring Boot Application |

Our application continues listening on port `8080` inside the container.

We mapped it to host port `8081` because the first container is already using host port `8080`.

This allows both versions to run simultaneously.

| Container | Image          | Host Port | Container Port |
|-----------|----------------|-----------|----------------|
| demo-v1   | docker-demo:v1 | 8080      | 8080           |
| demo-v2   | docker-demo:v2 | 8081      | 8080           |

We can verify both containers using:

```
dockerps
```

And test their endpoints independently:

```
curl http://localhost:8080/hello
```

```
curl http://localhost:8081/hello
```

## 5. Key Takeaways

1. A Docker image contains our application along with the runtime environment and dependencies required to execute it.
2. An already compiled Spring Boot JAR generally does not require a full JDK to run.
3. Choosing a JRE-based image instead of a JDK-based image can reduce the size of our production image.
4. `EXPOSE` documents the intended container port, whereas `p` actually publishes it on the host machine.
5. Multiple containers can listen on the same internal port as long as they are mapped to different host ports.